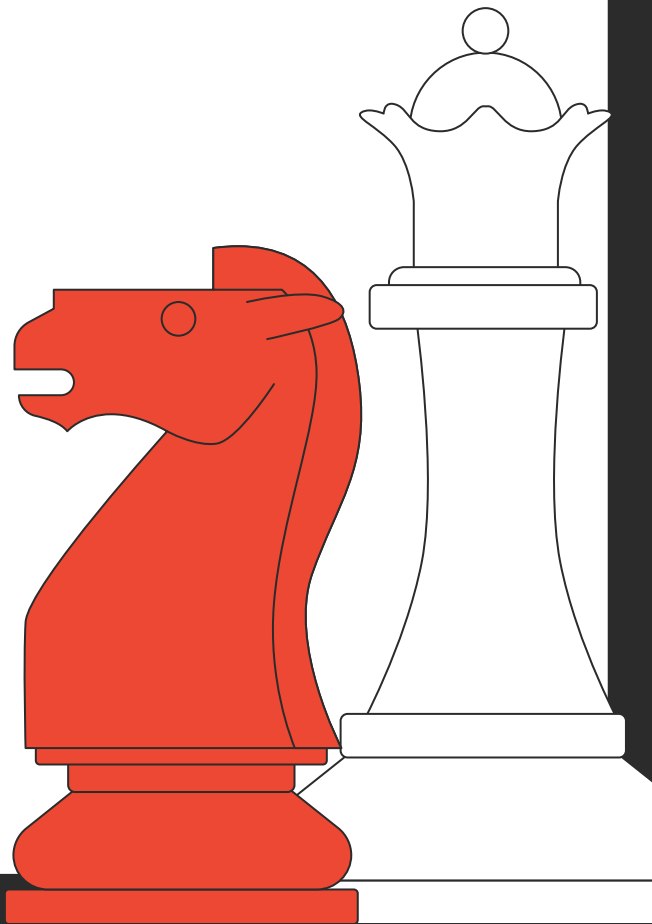
Chess AI Project (Group 11)

Names: Yash, Tianrun, Dhruv, Sasha, Charlie, Sween, Ayden, Yuxuan, Dehui

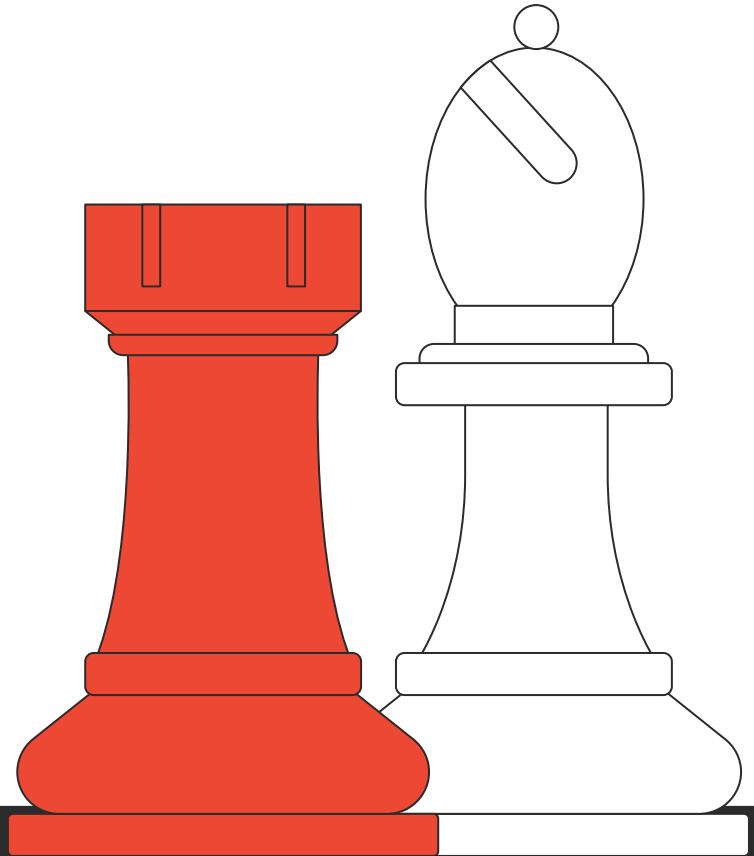


Presentation Overview

Introduction	Introduce the topic, why we chose it
Minimax	Minimax model discussion and results
ANN	ANN model discussion and results
Monte Carlo	Monte Carlo model discussion and results
Comparison	Compare and comment on the differences between results
Conclusion	Final thoughts

Introduction - Background and Motivation

- Chess has 10^{120} possible playable games
  - This number exceeds the amount of atoms in the visible **universe**
- Designing a chess engine requires careful consideration
  - Target Strength
  - Playstyle
  - Feature Engineering
- Different models allow us to compare and contrast algorithms for game playing AI



01

Minimax

Yash, Dhruv, Tianrun

(Minimax) Evaluation Function - Features

Features:

- Material
- King Safety
- Piece-Square Tables
- Pawn Structure
- Mobility
- Center Control
- Rook Files
- Bishop Pair
- Knight outposts

(Minimax) Evaluation Function - Problems

- Early stage Issues
  - Knight bias
  - Central pawn negligence
  - Inefficient execution

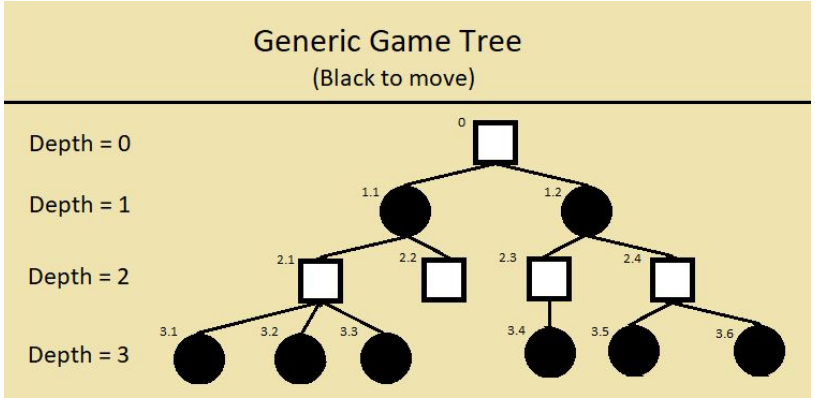
```
pawn_table = [
    0,   0,   0,   0,   0,   0,   0,   0,
    5,  10,  10, -25, -25,  10,  10,   5,
    5,  -5, -10,   0,   0, -10,  -5,   5,
    0,   0,   0,  32,  35,   0,   0,   0,
    5,   5,  10,  27,  27,  10,   5,   5,
   10,  10,  20,  30,  30,  20,  10,  10,
   50,  50,  50,  50,  50,  50,  50,  50,
    0,   0,   0,   0,   0,   0,   0,   0
]
```

- Resolving Issues
  - Stronger PST's
  - Move Ordering Enhancement
  - Stronger Center Bonus
  - Bitboard implementation

(Minimax) Search Algorithm - Basics

- Designed the minimax algorithm
  - Rational Agent Assumption
  - Recursive Search
  - Alpha-Beta Pruning
  - Call Evaluation Function

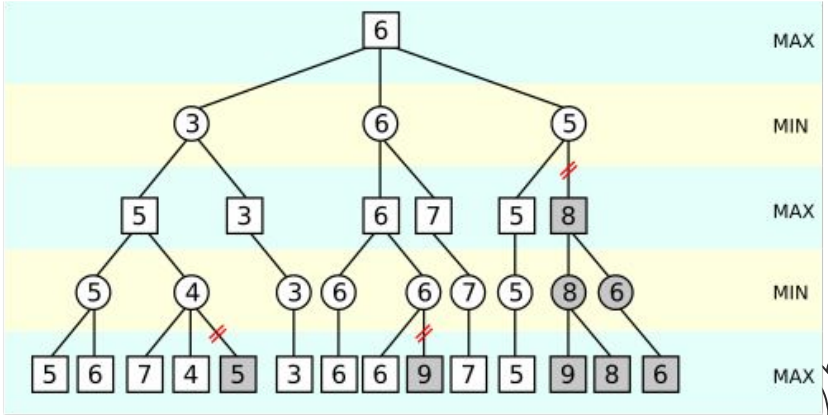
- Use returned scores from evaluation to play the move with the highest score/bonus
  - Calculated at depth m
  - Propagated back to current position



(Minimax) Search Algorithm - Optimizations

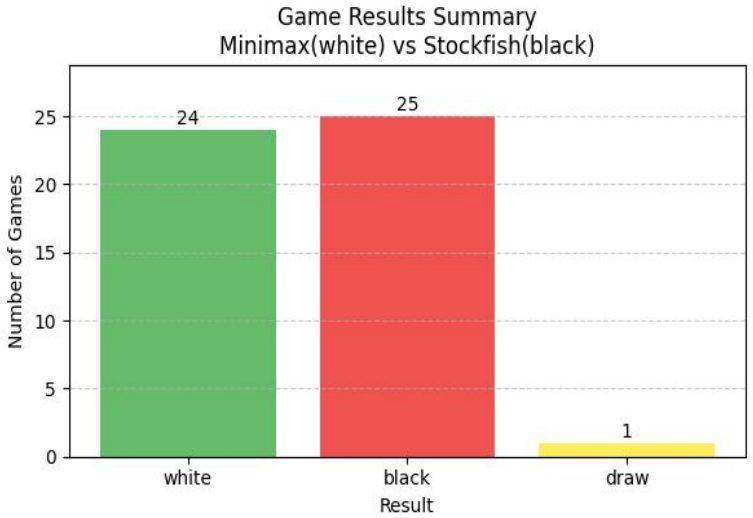
Added the following optimizations:

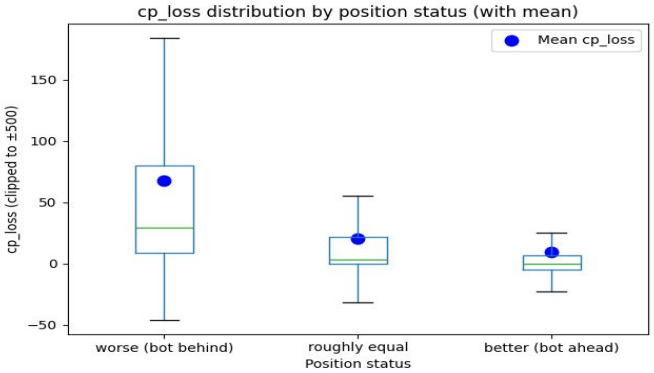
- Move Ordering
  - With Killer Moves/History
- Quiescence Search
- Static Exchange Evaluation(SEE)
- Iterative Deepening
- Transposition Table(TT)
- Null move pruning
- Aspiration Windows

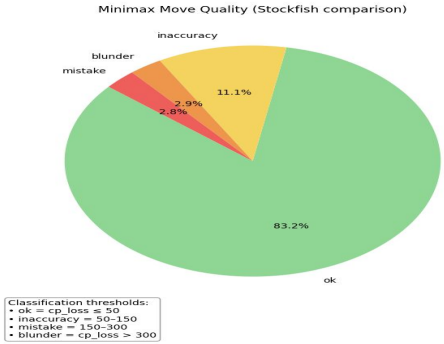


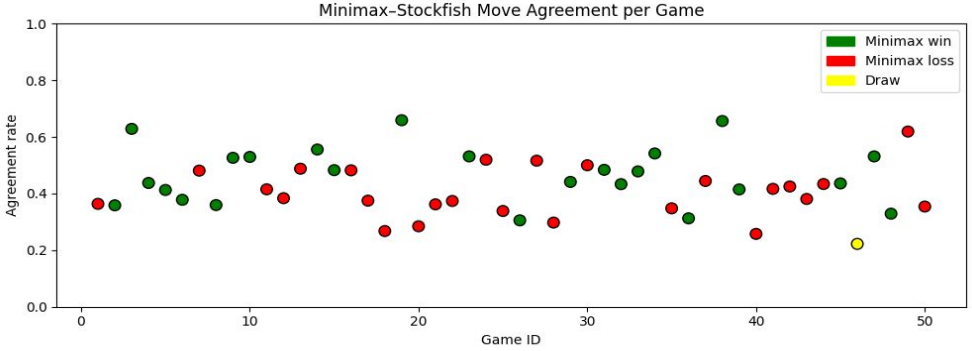
Minimax(White) vs 2000 Elo Stockfish(Black) Data and Results

Time per minimax move	13.577 seconds
Avg time per game	16.672 minutes
Median TT hit rate	44%
Mean SEE pruning rate	11%



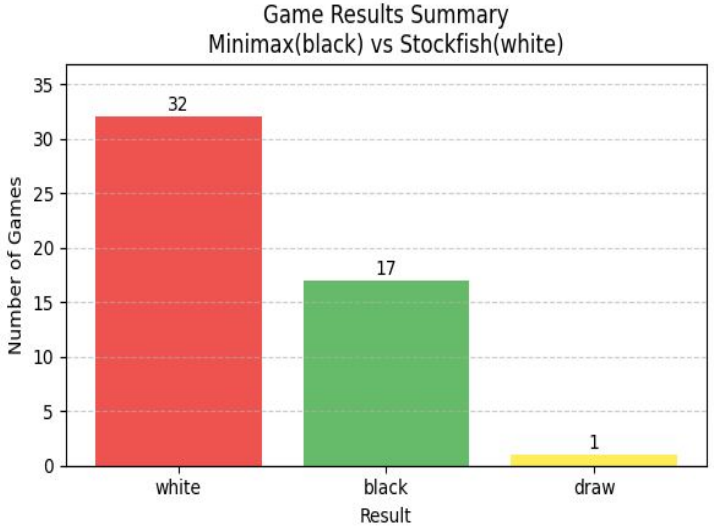

cp_loss distribution by position status (with mean)
Mean cp_loss
cp_loss (clipped to ±500)
150
100
50
0
−50
worse (bot behind)
roughly equal
better (bot ahead)
Position status

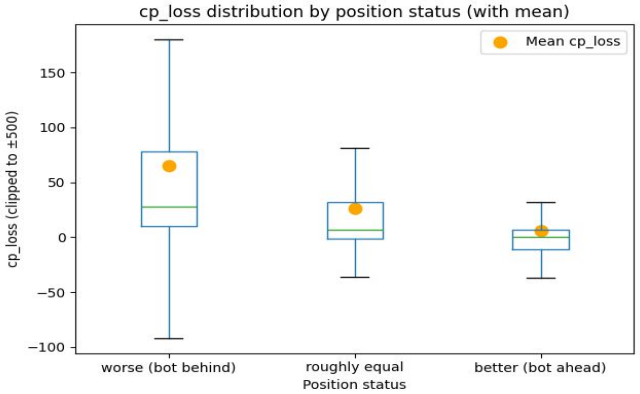

Minimax Move Quality (Stockfish comparison)
inaccuracy
blunder
mistake
11.1%
2.9%
2.8%
83.2%
ok
Classification thresholds:
• ok = cp_loss ≤ 50
• inaccuracy = 50–150
• mistake = 150–300
• blunder = cp_loss > 300

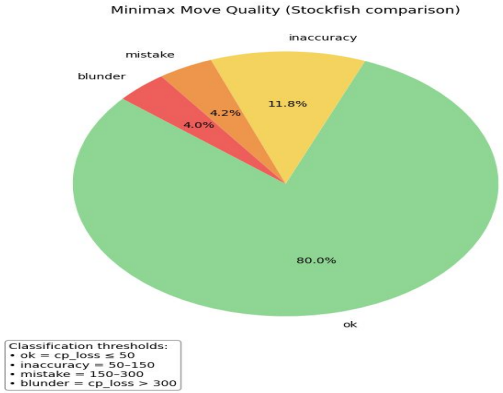

Minimax–Stockfish Move Agreement per Game
Minimax win
Minimax loss
Draw
Agreement rate
1.0
0.8
0.6
0.4
0.2
0.0
0
10
20
30
40
50
Game ID

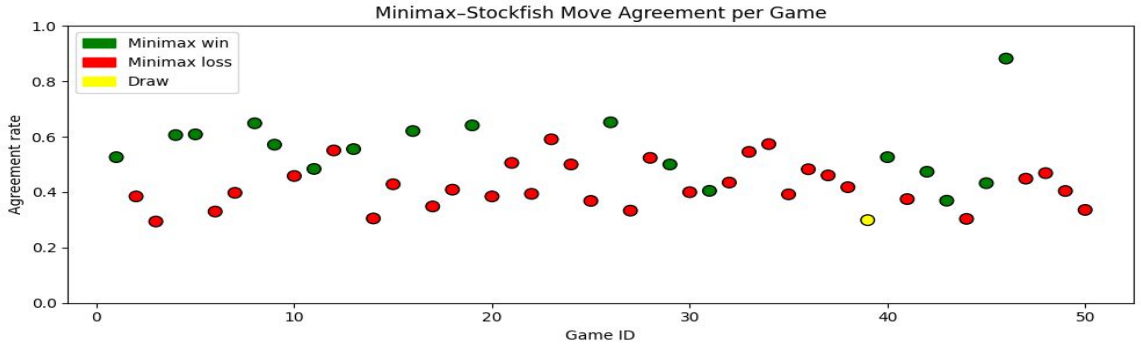
Minimax (Black) vs 2000 Elo Stockfish(White) Data and Results

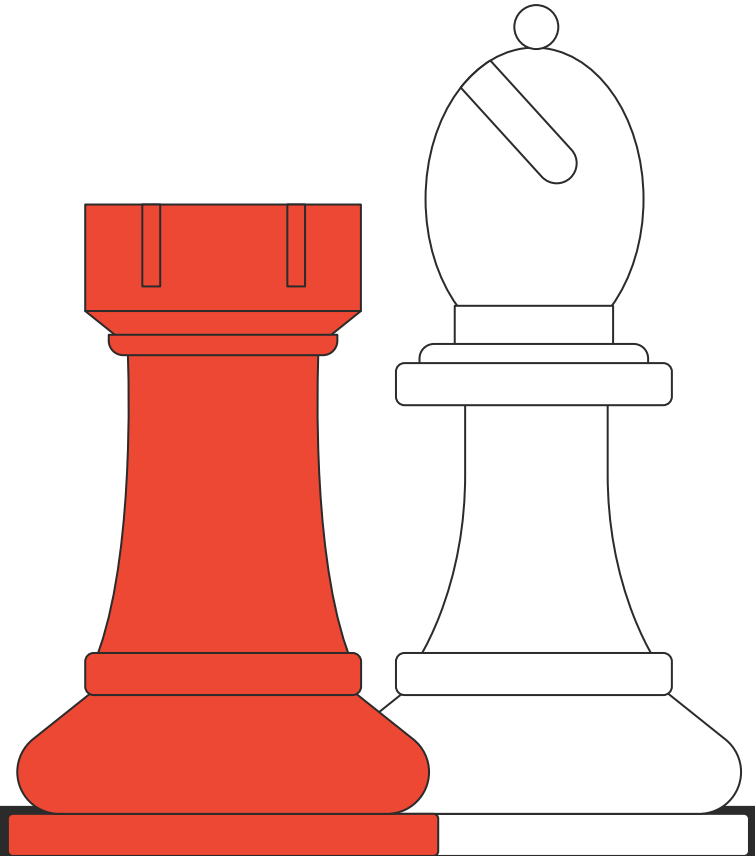
Time per minimax move	13.107 seconds
Avg time per game	13.875 minutes
Median TT hit rate	44%
Mean SEE pruning rate	14%




cp_loss distribution by position status (with mean)
Mean cp_loss
cp_loss (clipped to ±500)
150
100
50
0
−50
−100
worse (bot behind)
roughly equal
better (bot ahead)
Position status


Minimax Move Quality (Stockfish comparison)
inaccuracy
mistake
blunder
11.8%
4.2%
4.0%
80.0%
ok
Classification thresholds:
• ok = cp_loss ≤ 50
• inaccuracy = 50–150
• mistake = 150–300
• blunder = cp_loss > 300


Minimax–Stockfish Move Agreement per Game
Minimax win
Minimax loss
Draw
Agreement rate
1.0
0.8
0.6
0.4
0.2
0.0
0
10
20
30
40
50
Game ID



02

ANN

Charlie, Sween, Sasa

ANN - Data Generation

- Simulated ~2000 games = ~225,000 positions
- Using Stockfish with 40 randomized bots
  - 4 bots for each skill level
  - Random configs: threads (1/2/4), hash size (16/32/64 MB)
- Each game played between two random bots

Early mistake:

- Capped games at 70 moves/player
  - Lead to ~15% of games incorrectly being labeled as draws

ANN - Feature Engineering & EDA

Response Variable

- Originally tried game outcome
  - win/loss/draw
  - scaled
  - Scaled with tempo
- Switched to Stockfish evaluation value

EDA & Data Cleaning

- Removed checkmates encoded as ±9999
- Ignored near-equal positions (between -10 and 10)

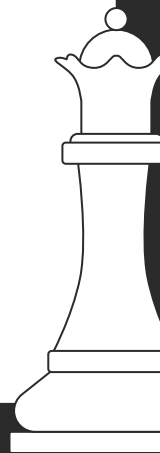
ANN - Feature Engineering & EDA Cont.

Feature Work

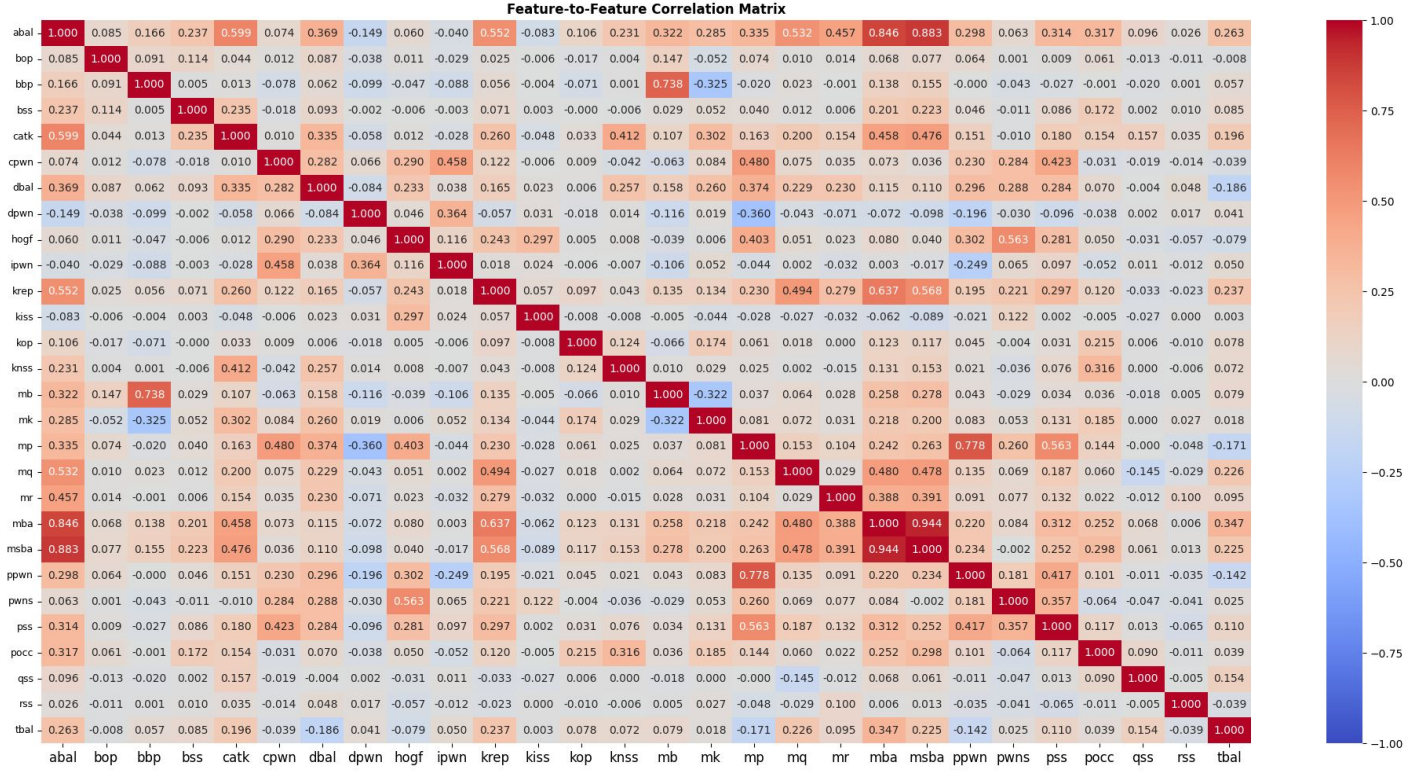
- Extracted 28 initial features, later expanded to 76
  - bishop pair
  - pawn shield
  - etc.
- Added “balanced” forms
  - white - black
  - black - white

Correlation Analysis

- Feature-to-feature
  - Removed redundant features
    - Isolated pawns
- Feature-to-response
  - Aggregated low-signal piece-square features
    - Piece square sums



Feature to Feature Correlation



ANN - Modeling & Performance

- Experiment: Linear Regression with L2 Penalty

GridSearch

- Hidden layer neurons: 512 > 256 > 128
- Learning rate: 0.001
- Dropout rate: 0.1
- Batch size: 256
- Activation function: ReLu

Training

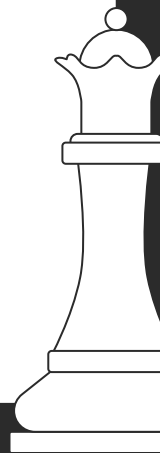
- 3-fold cross-validation
- Early stopping, trained over 20 epochs

Performance

- R^2 = 0.8971
- RMSE = 16.52
- MSE = 27.29

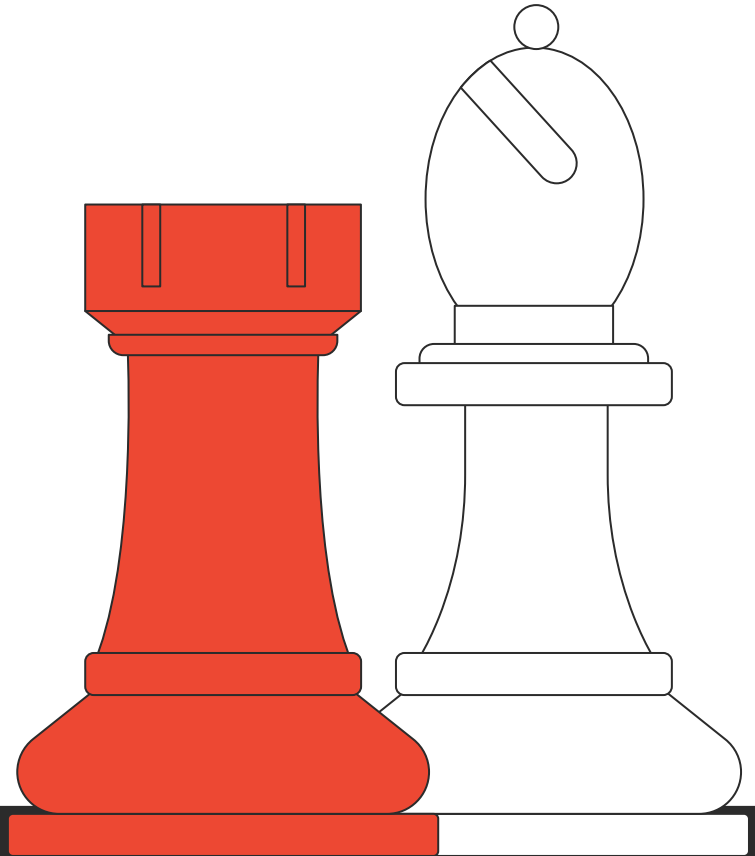
Gameplay with Minimax

- 0.5s move (d=2)
- 4s move (d=3)
- > 1min (d=4)



Grid Search Parameters Results

Neurons	Dropout	LR	Activation	MSE	RMSE	R^2
512	**0.1**	**0.001**	**relu**	**3.020**	**1.738**	**0.8854**
512	0.2	0.001	relu	3.054	1.748	0.8841
512	0.1	0.001	tanh	3.069	1.752	0.8836
512	0.2	0.001	tanh	3.081	1.755	0.8831
256	0.1	0.001	relu	3.089	1.758	0.8828
256	0.2	0.001	tanh	3.238	1.799	0.8772
256	0.1	0.001	tanh	3.196	1.788	0.8787
256	0.1	0.01	relu	3.278	1.811	0.8756
512	0.1	0.01	tanh	3.395	1.843	0.8712
256	0.1	0.01	tanh	3.383	1.839	0.8717
512	0.2	0.01	tanh	3.446	1.856	0.8693
256	0.2	0.01	tanh	3.438	1.854	0.8696
512	0.2	0.01	relu	3.746	1.935	0.8579
256	0.2	0.01	relu	3.845	1.961	0.8541



03
Monte Carlo

Ayden, Dehui, Yuxuan

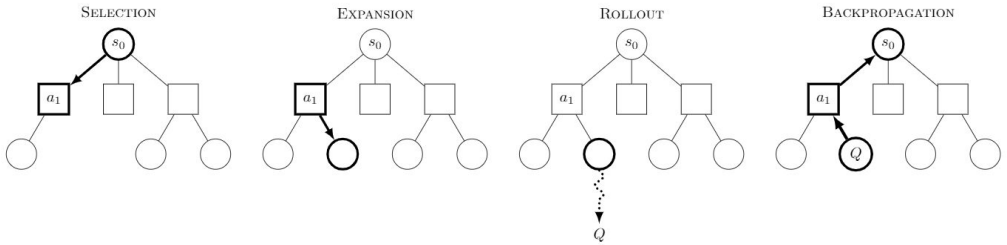
MCTS - Usage in Chess

Selection – choose child using UCT

Expansion – add one new child node

Simulation – estimate value of the position

Backpropagation – update visits & values



MCTS - Overall Statistics (vs itself)

game	winner	moves	time(sec)	timePerMove	avgSimDepth	numRootSim	avgBoardEvalScore	numFoundWinningMoveSets
1	white	51	216.4	2.1996339559555054	73.26875	799.0	0	388.5
2	*	300	1145.7	3.4448204040527344	95.35125	799.0	0	11.5
3	*	300	1386.6	3.4076679944992065	104.609375	799.0	0	4.0
4	*	300	1100.0	1.9571480751037598	57.709375	799.0	0	0.0
5	tie	255	884.4	0.7912745475769043	19.771875	799.0	0	1.0
6	tie	230	816.3	1.0699118375778198	34.5075	799.0	0	1.5
7	tie	196	694.0	1.2012169361114502	38.7325	799.0	0	0.0
8	*	300	1125.4	3.9501932859420776	126.908125	799.0	-1218	12.5
9	tie	236	701.5	0.6797255277633667	16.21875	799.0	0	2.0
10	*	300	1155.3	3.6457656621932983	120.765	799.0	17	0.0
avg	-	246.8	922.6	2.2347358226776124	68.78425	799.0	-120.1	42.1

MCTS - Overall Statistics (vs Stockfish)

game	winner	moves	time(sec)	timePerMove	avgSimDepth	numRootSim	avgBoardEvalScore	numFoundWinningMoveSets
1	black	68	139.0	3.742037057876587	136.32	799	-1440	24
2	black	44	86.9	3.5286753177642822	139.03625	799	-961	13
3	black	10	20.6	3.9801321029663086	140.73875	799	-565	32
4	black	40	83.4	3.7510569095611572	141.73875	799	918	25
5	black	26	52.2	3.8443453311920166	141.61	799	152	24
6	black	64	126.1	3.7172727584838867	137.81125	799	-511	9
7	black	50	99.0	3.7233338356018066	138.63	799	-93	7
8	black	70	132.4	3.2560627460479736	133.96875	799	0	4
9	black	116	220.9	3.379620313644409	130.36125	799	-1791	12
10	black	50	108.4	3.935218334197998	135.48	799	-591	2
avg	-	53.8	106.9	3.6857754707336428	137.5695	799.0	-488.2	15.2

MCTS - Optimizations: Three types

Remember

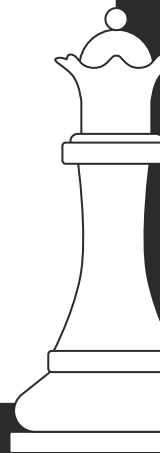
Remember scores of each board state between moves

Look at Tried moves

Swap between simulated and unsimulated moves

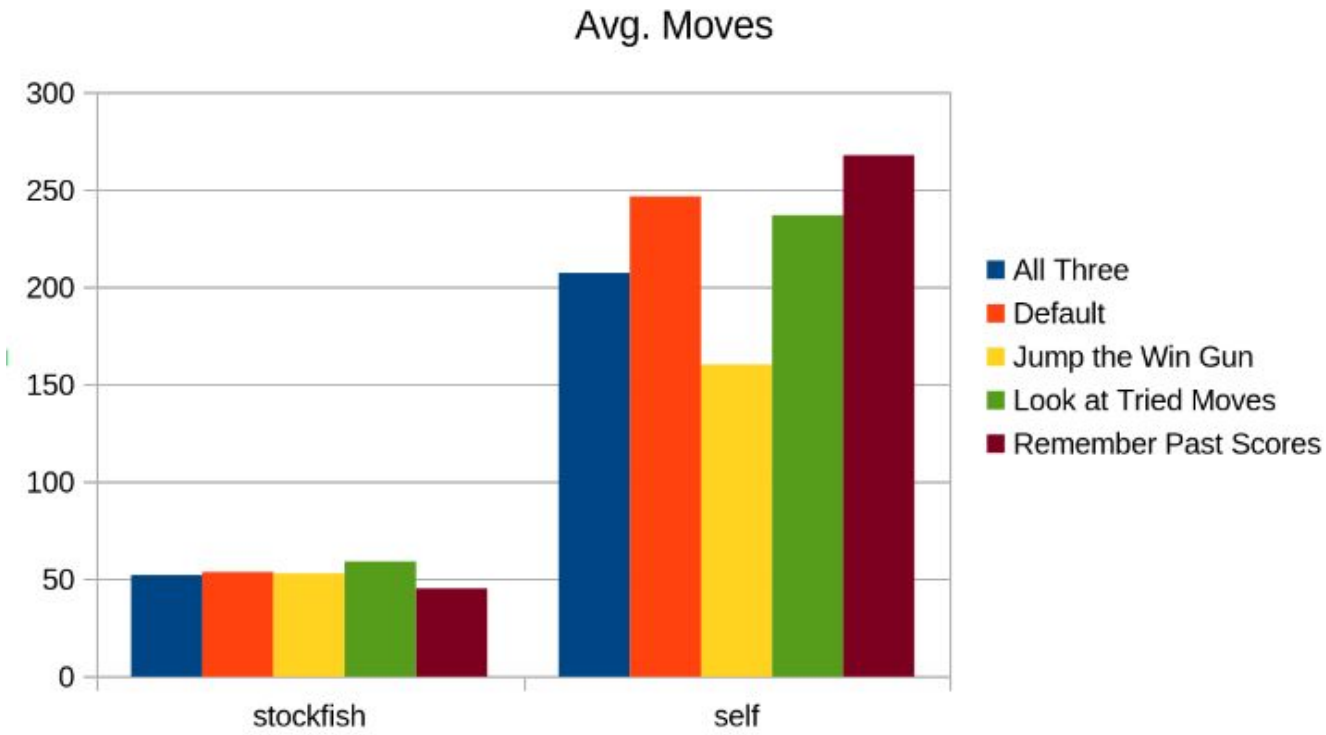
Jump the Win Gun

Stop simulation process if the simulation finds a winning state



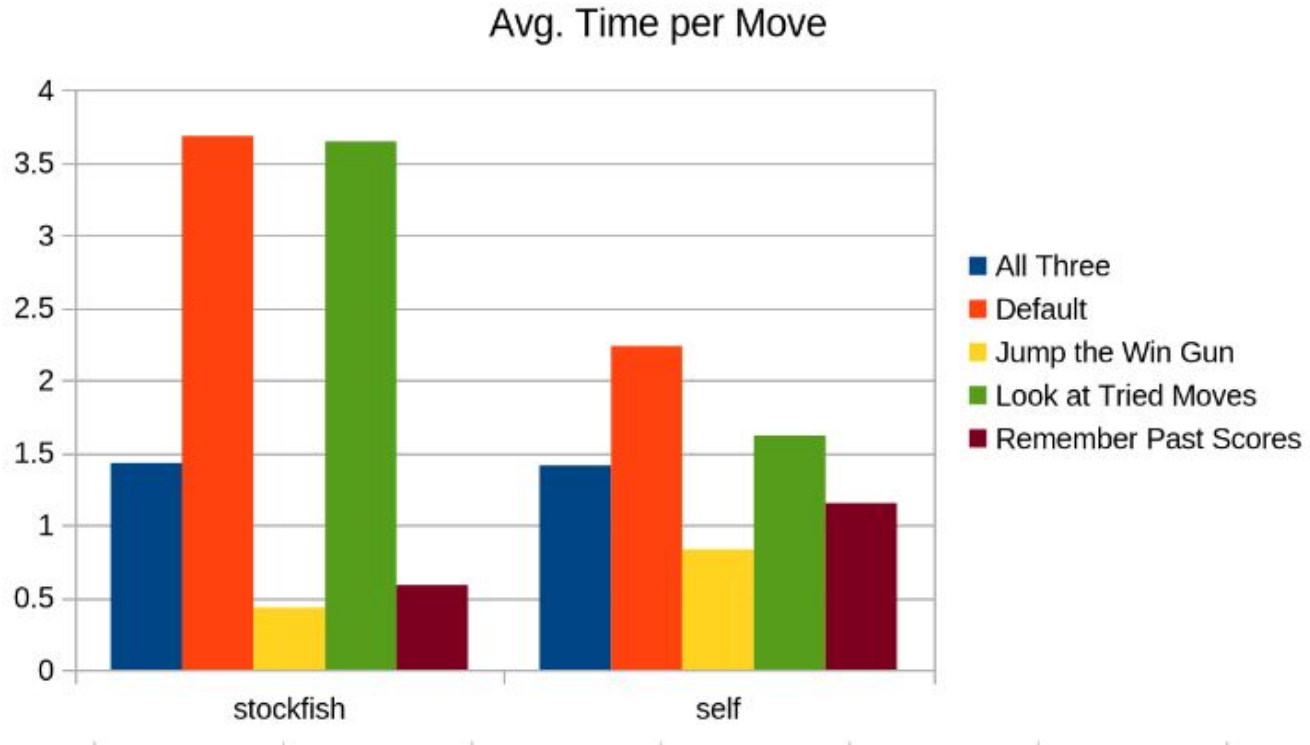
MCTS - Optimizations

Average Moves for each type



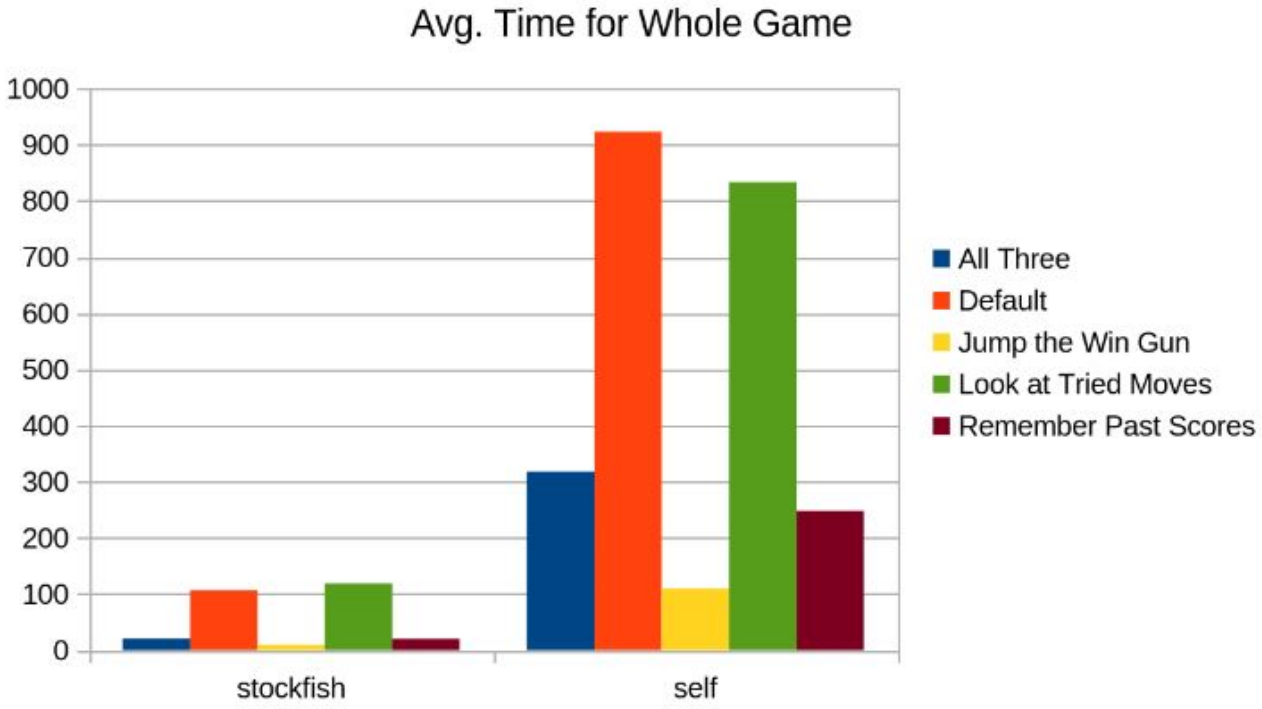
MCTS - Optimizations

Average Time Per Move



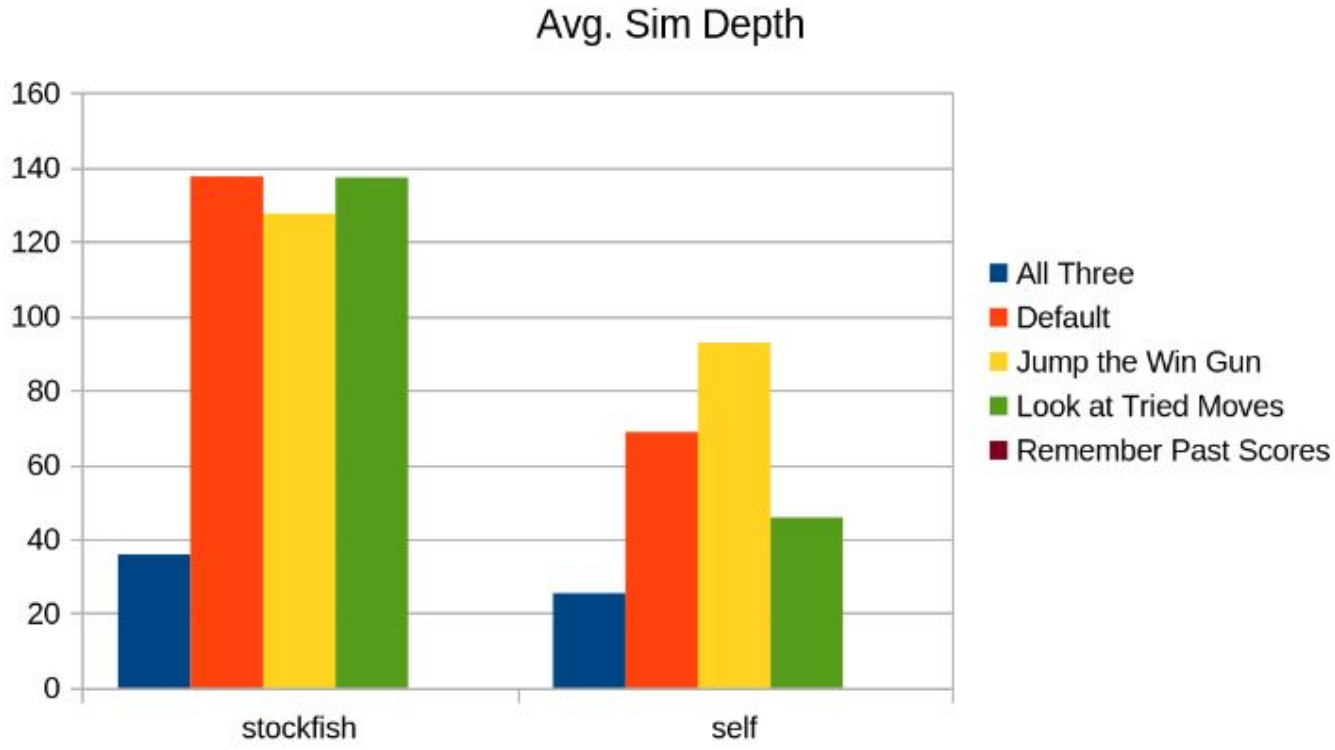
MCTS - Optimizations

Average Time to Complete Game



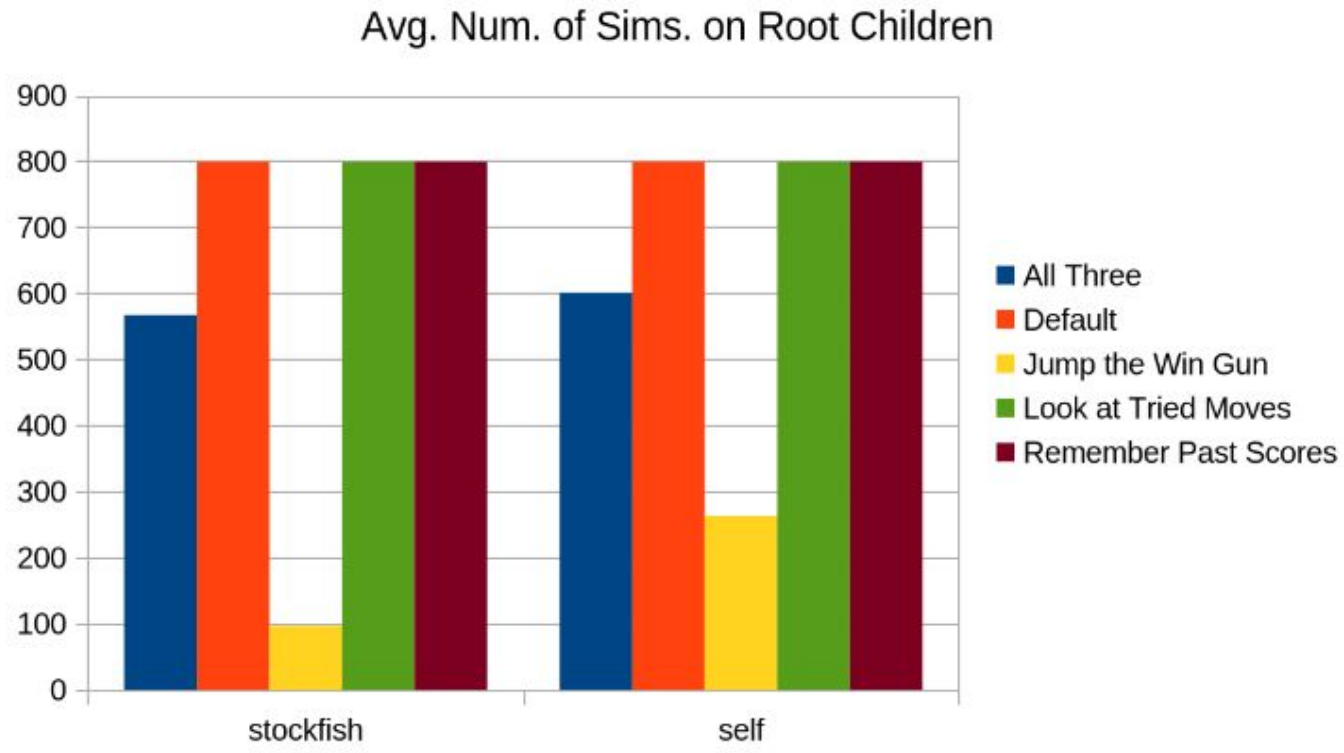
MCTS - Optimizations

Average Simulation Depth



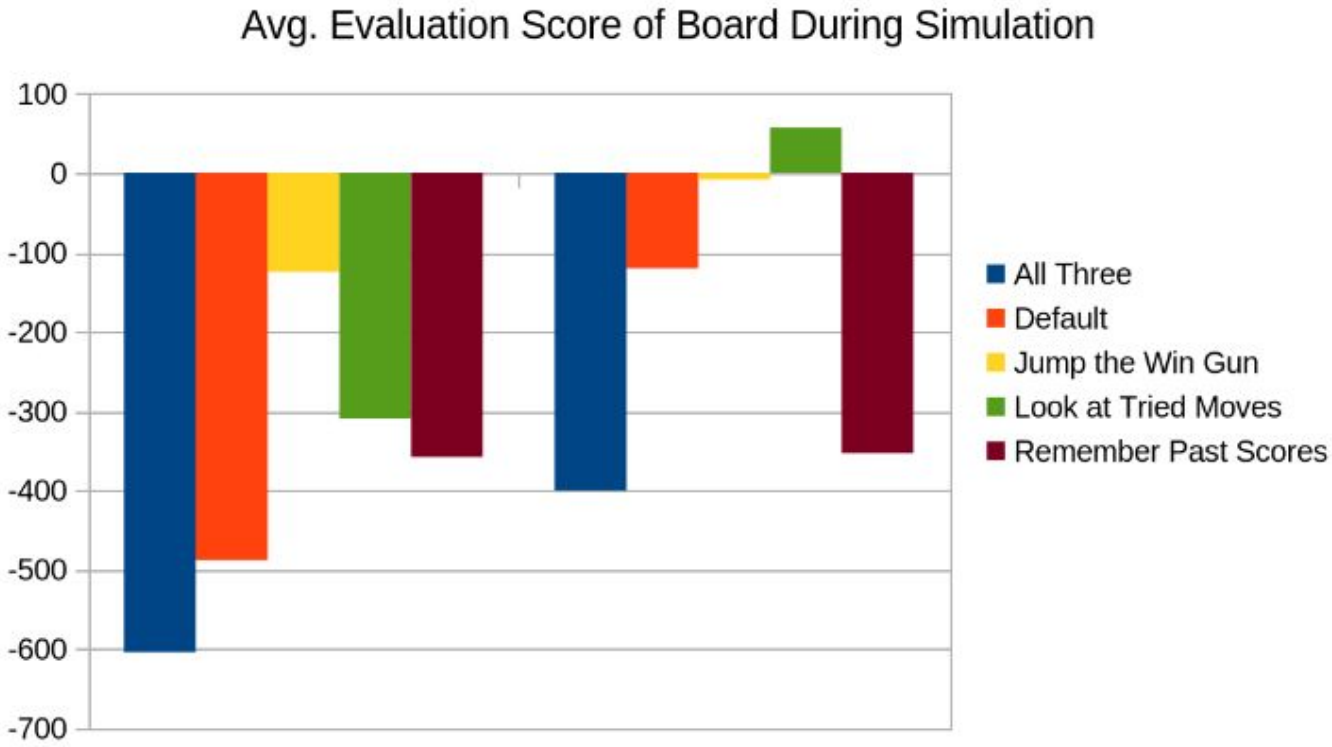
MCTS - Optimizations

Average Number of Simulations on Root's Children



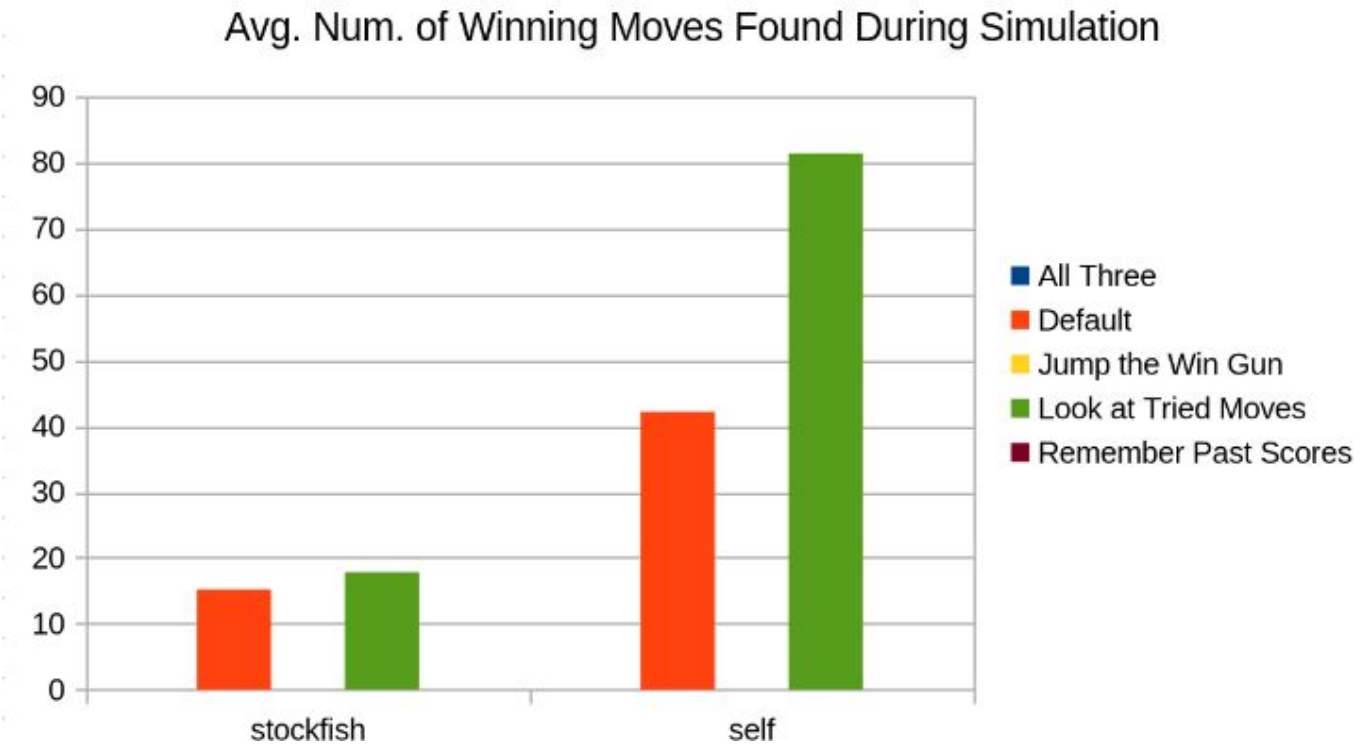
MCTS - Optimizations

Average Evaluation Score of Board During SImulation



MCTS - Optimizations

Average Number of Winning Moves Found During Simulation



Conclusion

- **Minimax**
  - BFS-like approach
  - Similar to human
  - Most optimal for chess
- **ANN**
  - Accurate eval
  - Slow with Gameplay Algorithms
- **Monte Carlo**
  - DFS-like approach
  - Non-human methodology
  - Weak due to randomness